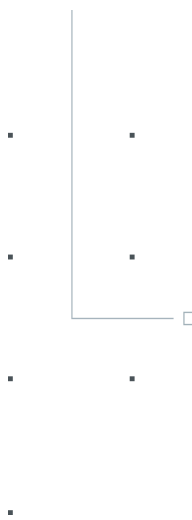


FIAP

NABA



MBA em Engenharia de Dados

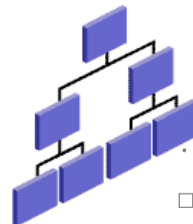


Criação de Índice para a melhoria de desempenho



Um índice:

- É um objeto de esquema
- É usado pelo SGBDR para acelerar a recuperação de linhas usando um ponteiro
- Pode reduzir o I/O de disco por um método de acesso de caminho rápido para localizar os dados com mais velocidade.
- É independente da tabela que indexa
- É usado e mantido automaticamente pelo SGBDR.

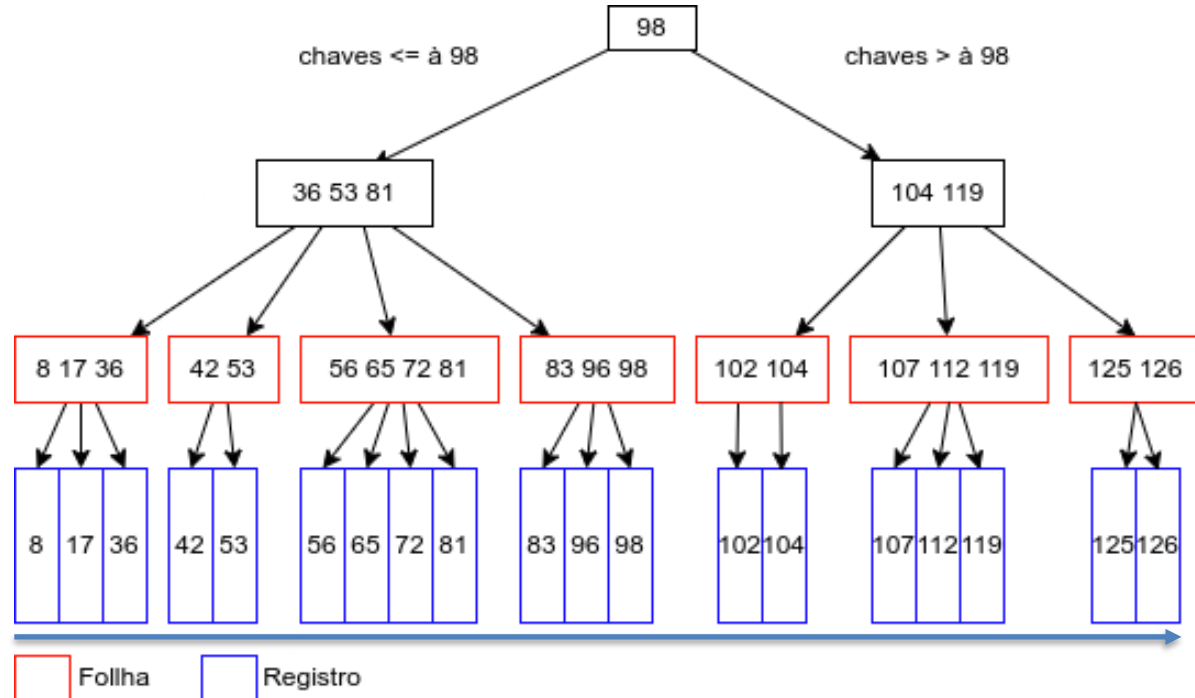


Índices

index	company_num	ad_num	hit_fee
13	13	48	0.1
14	23	49	0.01
17	17	52	0.12
23	13	55	0.34
24	14	64	0.01
	24	68	0.04
	13	77	0.07
	23	109	0.01

Índices

- São organizado em estrutura de árvore B



Como Criar Índices?

- Automaticamente: Um índice exclusivo é criado automaticamente quando você define uma constraint de chave PRIMARY ou UNIQUE em uma definição de tabela.



- Manualmente: Os usuários podem criar índices não exclusivos em colunas para acelerar o acesso às linhas.



Criando um Índice

- Crie um índice em uma ou mais colunas:

```
CREATE INDEX index  
ON table (column[, column]...);
```

- Aumente a velocidade de acesso da consulta à coluna LAST_NAME da tabela EMPLOYEES:

```
CREATE INDEX emp_last_name_idx  
ON          employees(last_name);
```


Diretrizes para a Criação de um Índice

Crie um índice quando:	
☒	Uma coluna contiver uma grande faixa de valores
☒	Uma coluna contiver um grande número de valores nulos
☒	Uma ou mais colunas forem usadas em conjunto com frequência em uma cláusula <code>WHERE</code> ou em uma condição de
☒	A tabela for grande e for esperado que a maioria das consultas recupere menos de 2% a 4% das linhas da tabela
Não crie um índice quando:	
☒	As colunas não forem usadas com frequência como uma condição na consulta
☒	A tabela for pequena ou for esperado que a maioria das consultas recupere mais de 2% a 4% das linhas da tabela
☒	A tabela for atualizada com frequência
☒	As colunas indexadas forem referenciadas como parte de uma expressão

Vamos Praticar

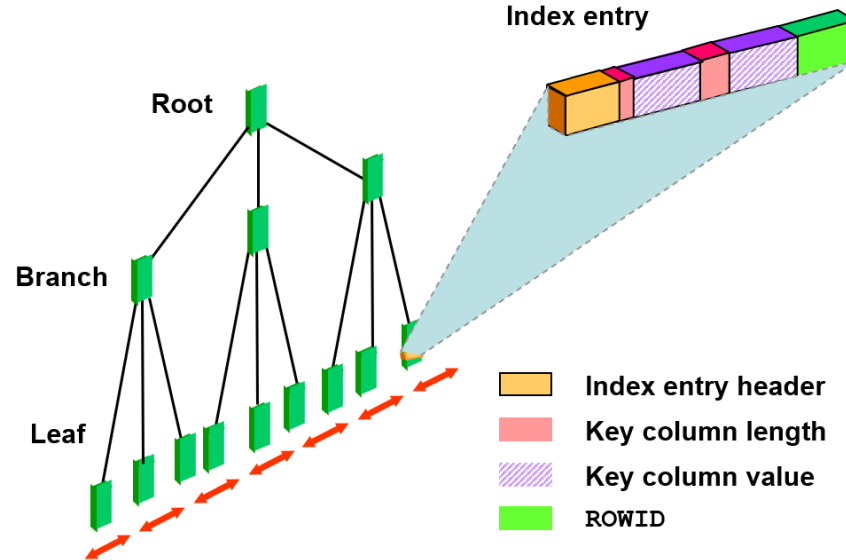
Arquivo: 04 - Pratica INDEXES.pdf

- **PARTE I** – Uso de índice para otimização de consultas
- **PARTE II** – Otimizando criação de índice em grandes tabelas (grande volume de dados)

Apenas partes I e II

Tipos de Índices

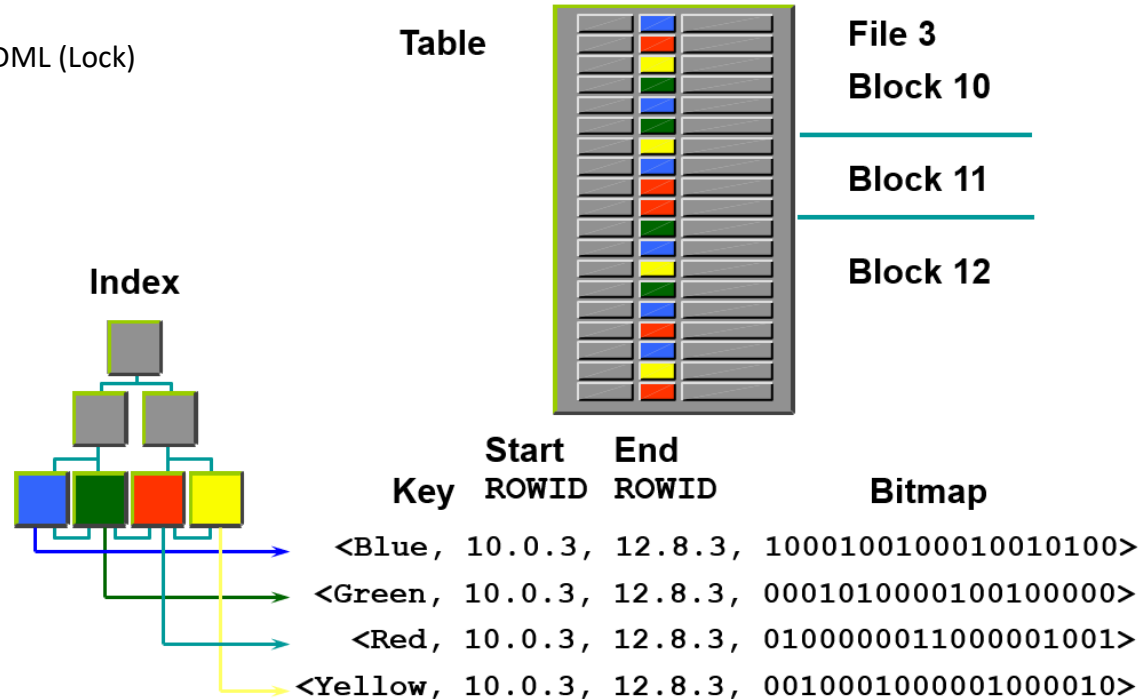
Índice B-tree



Tipos de Índices

Índice Bitmap

- Apenas uma sessão faz DML (Lock)
- Tipicamente ETL
- DW ou Bases Analíticas

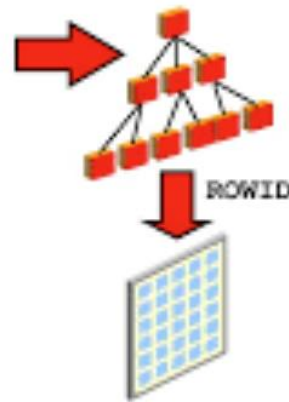


IOT - Index Organized Tables

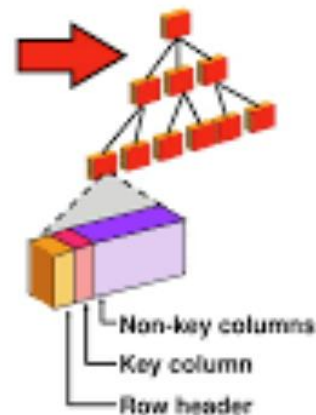
```
CREATE TABLE locations
(id      NUMBER(10),
description VARCHAR2(50) NOT NULL,
map      BLOB,
CONSTRAINT pk_locations PRIMARY KEY (id)
)
ORGANIZATION INDEX;
```

- Acesso rápido por chave primária
- Redução de redundância de armazenamento
- Tabelas com chaves primárias extensa
- Consultas e DML baseadas em chave primária
- Possuem restrições ao usar features dos BD e questões de fragmentação em certas condições de DML

Índice padrão



IOT



Vamos Praticar

Arquivo: 04 - Pratica INDEXES.pdf

- **PARTE III – Índice B-tree x Bitmap**

Complete a parte I
antes de fazer esta
prática

Function-Based Indexes

- O uso de funções em colunas usadas na cláusula WHERE pode levar a um FULL TABLE SCAN, impactando na performance do comando SQL
- Mesmo que a coluna contenha um índice, este não consegue ser utilizado pelo otimizador de consulta.
- Uma solução é criar um índice baseado em função

Function-Based Indexes

```
SQL> SET AUTOTRACE TRACEONLY EXPLAIN
SQL> SELECT * FROM HR.EMPLOYEES WHERE MANAGER_ID > 200;
```

Execution Plan

Plan hash value: 621391157

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		1	69	2 (0)	00:00:01
1	TABLE ACCESS BY INDEX ROWID BATCHED	EMPLOYEES	1	69	2 (0)	00:00:01
* 2	INDEX RANGE SCAN	EMP_MANAGER_IX	1		1 (0)	00:00:01

➔ Note o uso do índice que existe na coluna MANAGER_ID

... Mas o que aconteceria se o comando usasse NVL(MANAGER_ID,0) > 200 ?

Function-Based Indexes

```
SQL> SELECT * FROM HR.EMPLOYEES WHERE NVL(MANAGER_ID,0) > 200;
```

Execution Plan

Plan hash value: 1445457117

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		1	69	3 (0)	00:00:01
* 1	TABLE ACCESS FULL	EMPLOYEES	1	69	3 (0)	00:00:01

... O Uso da função forçou a um FULL TABLE SCAN. A solução neste caso é criar um índice para NVL(MANAGER_ID,0)

Function-Based Indexes

- **CREATE INDEX HR.IX_NVL_MANAGERID ON HR.EMPLOYEES (NVL(MANAGER_ID,0))
ONLINE COMPUTE STATISTICS;**

```
SQL> CREATE INDEX HR.IX_NVL_MANAGERID ON HR.EMPLOYEES (NVL(MANAGER_ID,0))
2 ONLINE COMPUTE STATISTICS;
```

Index created.

```
SQL> SELECT * FROM HR.EMPLOYEES WHERE NVL(MANAGER_ID,0) > 200;
```

Execution Plan

Plan hash value: 2969834138

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		5	345	2 (0)	00:00:01
1	TABLE ACCESS BY INDEX ROWID BATCHED	EMPLOYEES	5	345	2 (0)	00:00:01
* 2	INDEX RANGE SCAN	IX_NVL_MANAGERID	1		1 (0)	00:00:01

INDEXES

- Avalie quando ou não criar um índice
- Se for criar um índice, avalie as opções de indexação do SGBD-R
- Índices podem necessitar de reorganização após algum tempo
- O correto uso de índices e um SQL bem escrito são pontos chaves para otimização de acesso aos dados



Removendo um Índice

- Para remover um índice do dicionário de dados, use o comando `DROP INDEX`:

```
DROP INDEX index;
```

- Remova o índice `UPPER_LAST_NAME_IDX` do dicionário de dados:

```
DROP INDEX emp_last_name_idx;
```

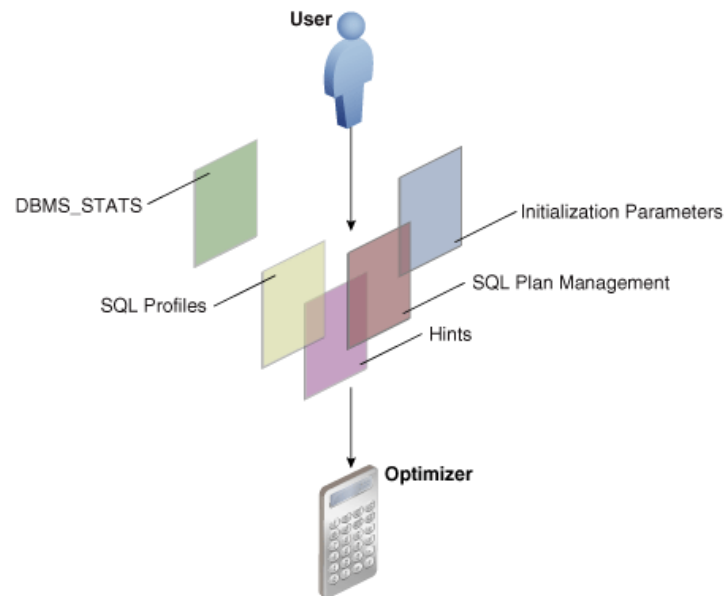
- Para eliminar um índice, você precisa ser o proprietário dele ou ter o privilégio `DROP ANY INDEX`.

Influenciando o Otimizador

- O otimizador de consulta se baseia em várias regras e **estatísticas** sobre como estão os dados armazenados
- As configurações padrão do otimizador são suficientes para a maioria das operações, mas não para todas.
- As configurações padrão do otimizador são suficientes para a maioria das operações, mas não para todas.
- Em alguns casos, você pode ter informações que são desconhecidas pelo otimizador ou precisar ajustar o otimizador para um tipo específico de instrução ou carga de trabalho.
- Nessas situações, influenciar o otimizador pode resultar em um desempenho melhor.

Técnicas para Influenciar o Otimizador

- Você pode influenciar o otimizador usando várias técnicas, incluindo perfis de SQL (SQL profile), gerenciamento de plano SQL, parâmetros de inicialização e hints.
- A figura ilustra as principais técnicas para influenciar o otimizador.



Hints

- Você pode usar hints para influenciar o modo do otimizador, a transformação de consultas, o caminho de acesso, a ordem de junção e os métodos de junção.

Hints

- Em um ambiente de teste, hints são valiosos para avaliar o desempenho de um caminho de acesso específico.
- Por exemplo, você pode saber que um índice é mais seletivo para determinadas consultas, resultando em um plano de execução melhor.

Hints

- A desvantagem dos hints é o código extra necessário para gerenciar, verificar e controlar. Os hints foram introduzidos no Oracle7, quando os usuários tinham poucas alternativas caso o otimizador gerasse planos mais eficientes.

Hints

- Como mudanças no banco de dados e no ambiente do host podem tornar os hints obsoletos ou ter consequências negativas, uma boa prática é testar usando hints, mas utilizar outras técnicas para gerenciar os planos de execução.

Exemplo: Otimizador de Consulta em ação

```
SQL> SET AUTOTRACE TRACEONLY EXPLAIN
SQL> SELECT * FROM HR.EMPLOYEES WHERE MANAGER_ID > 200;
```

Execution Plan

Plan hash value: 621391157

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		1	69	2 (0)	00:00:01
1	TABLE ACCESS BY INDEX ROWID BATCHED	EMPLOYEES	1	69	2 (0)	00:00:01
* 2	INDEX RANGE SCAN	EMP_MANAGER_IX	1		1 (0)	00:00:01



```
SQL> SELECT * FROM HR.EMPLOYEES WHERE MANAGER_ID > 10
```

Execution Plan

Plan hash value: 1445457117

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		106	7314	3 (0)	00:00:01
* 1	TABLE ACCESS FULL	EMPLOYEES	106	7314	3 (0)	00:00:01



Exemplo: Otimizador de Consulta em ação

```
SQL> SELECT /*+ FULL(EMPLOYEES) */ * FROM HR.EMPLOYEES WHERE MANAGER_ID > 200;
```

Execution Plan

Plan hash value: 1445457117

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		1	69	3 (0)	00:00:01
* 1	TABLE ACCESS FULL	EMPLOYEES	1	69	3 (0)	00:00:01

Apenas como exemplo, usamos o hint FULL para forçar um FULL TABLE SCAN na tabela EMPLOYEES. Neste caso o plano é pior que usar o índice pois o número de registros retornado é pequeno

Exemplo: Otimizador de Consulta em ação

```
SQL> SELECT /*+ INDEX (EMPLOYEES EMP_MANAGER_IX) */ * FROM HR.EMPLOYEES WHERE MANAGER_ID > 10;
```

Execution Plan

Plan hash value: 621391157

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		106	7314	8 (0)	00:00:01
1	TABLE ACCESS BY INDEX ROWID BATCHED	EMPLOYEES	106	7314	8 (0)	00:00:01
* 2	INDEX RANGE SCAN	EMP_MANAGER_IX	106		1 (0)	00:00:01

... Use o hint INDEX para forçar a utilização de um índice da tabela

➔ A melhor prática é manter as estatísticas sempre atualizadas e evitar uso de Hints nos códigos fontes das aplicações

- **schema:** DBMS_STATS.GATHER_SCHEMA_STATS ('HR');
- **tabela:** DBMS_STATS.GATHER_TABLE_STATS ('HR', 'EMPLOYEES');

External Tables

- As tabelas externas são como as tabelas regulares, mas apresentam algumas exceções:
- As tabelas externas são criadas com o comando

```
CREATE TABLE ...  
ORGANIZATION EXTERNAL".
```
- **Os dados estão armazenados fora do banco de dados em arquivos do SO, portanto, a organização é EXTERNA.**

External Tables: Exemplo

1. Usando um editor de texto, vamos criar dois arquivos que contém os dados

/home/oracle/Countries1.txt

```
ENG,England,English  
SCO,Scotland,English  
IRE,Ireland,English  
WAL,Wales,Welsh
```

/home/oracle/Countries2.txt

```
FRA,France,French  
GER,Germany,German  
USA,Unites States of America,English
```

External Tables: Exemplo

2. É necessário ter um objeto DIRECTORY no banco de dados que faz o mapeamento do “diretório” no banco de dados para um local físico no servidor, onde os arquivos existem.

```
sqlplus / as sysdba
```

```
CREATE OR REPLACE DIRECTORY ext_tab_data AS '/home/oracle';
```

➔ Note que “/home/oracle” é exatamente onde existem os arquivos textos criados anteriormente

External Tables: Exemplo

3. Vamos criar a uma external table chamada countries_ext

```
CREATE TABLE HR.countries_ext (  
  country_code  VARCHAR2(5),  
  country_name  VARCHAR2(50),  
  country_language VARCHAR2(50)  
)  
ORGANIZATION EXTERNAL (  
  TYPE ORACLE_LOADER  
  DEFAULT DIRECTORY ext_tab_data  
  ACCESS PARAMETERS (  
    RECORDS DELIMITED BY NEWLINE  
    FIELDS TERMINATED BY ','  
    MISSING FIELD VALUES ARE NULL  
    (  
      country_code  CHAR(5),  
      country_name  CHAR(50),  
      country_language CHAR(50)  
    )  
  )  
  LOCATION ('Countries1.txt','Countries2.txt')  
)  
PARALLEL 2  
REJECT LIMIT UNLIMITED;
```

External Tables: Exemplo

4. Acessando dados

```
SELECT * FROM HR.countries_ext;
```

```
SQL> select * from HR.countries_ext;
```

COUNT	COUNTRY_NAME	COUNTRY_LANGUAGE
FRA	France	French
GER	Germany	German
USA	Unites States of America	English
ENG	England	English
SCO	Scotland	English
IRE	Ireland	English
WAL	Wales	Welsh

```
7 rows selected.
```

External Tables

- Muito úteis para ingestão e/ou integração de dados
- Usadas em processos ETL
- Várias opções de configuração e formatos dos arquivos
- São apenas para leitura
- ➔ Não permitem utilização de índices

OBRIGADO



<https://www.linkedin.com/in/lucianoimelo>

FIAP MBA⁺

Copyright © 2025 | Professor. Aleandre Barcelos & Luciano Melo
 Todos os direitos reservados. Reprodução ou divulgação total ou parcial deste documento, é expressamente
 proibido sem consentimento formal, por escrito, do professor/autor.

FIAP